How to use CMake And Clang-Tidy:

By Shachar Tsrafati

Note: This guide will show in Windows but the step should be the same in mac

Before I will explain how to use CMake and TIdy, I will explain what is CMake and TIdy

So what is CMake ?

CMake is **not a compiler**. It is a **build system generator**.

You may have already written complex **Makefiles** in the System Programming 1 course, or manually configured projects in IDEs such as **Visual Studio** or **CLion**.

Instead of writing build files manually, CMake allows you to define the project configuration in a simple file. From this configuration, CMake automatically generates the required build system (for example **Makefiles**, **Ninja**, or **Visual Studio project files**).

In the root directory of your project, you need to create a configuration file called:

```
CMakeLists.txt
```

It is **very important that the filename is exactly this**, otherwise CMake will not recognize it.

Some IDEs such as **CLion** can generate this file automatically, and in many development environments you can create it by **right-clicking the project folder and selecting "New → CMakeLists.txt"**.

Why CMake?

• **Cross-platform support:**
CMake is a cross-platform build system. You define the project configuration once, and CMake can generate build files for different operating systems such as **Linux, Windows, and macOS**.

• **Library management:**
CMake makes it easier to work with external libraries. You can specify that your project depends on a certain library, and **CMake** will help locate it on the system and link it to your project.

• **Automation:**
When a new source file is added to the project, you usually only need to add it to the source list in **CMakeLists.txt**. CMake then takes care of generating the necessary build rules automatically.

Why Clang-Tidy?

Clang-Tidy is a static analysis tool for C and C++ code. It analyzes the source code without running the program and helps detect potential problems.

We use Clang-Tidy for several reasons:

• **Find bugs early:**
Clang-Tidy can detect common programming mistakes, undefined behavior, and potential runtime errors before the program is executed.

• **Improve code quality:**
It checks the code against best practices and modern C++ guidelines, helping developers write safer and cleaner code.

• **Enforce coding standards:**
Teams can use Clang-Tidy to ensure that all code follows the same style and rules.

• **Automatic suggestions and fixes:**
Some issues detected by Clang-Tidy can be automatically fixed or improved by the tool.

By using Clang-Tidy regularly during development, programmers can catch issues earlier and maintain higher code quality.

How to use CMake ?

Very simple you need to know a few commands. Lets assume your project looks like this

```
MyProject/
├── CMakeLists.txt   # main configuration file
├── main.cpp         # application source code
├── my_lib.cpp       # library implementation
└── my_lib.hpp       # library header file
```

And these are our files:

my_lib.hpp:

```
#ifndef MY_LIB_HPP
#define MY_LIB_HPP
void print_hello();
#endif
```

---

my_lib.cpp:

```
#include "my_lib.hpp"

#include <iostream>

void print_hello() {

   std::cout << "Hello from our custom library!" << std::endl;

}
```

So if we need to make a makefile like old time i will look like this:

Makefile

```
CXX = g++

CXXFLAGS = -std=c++17

all: my_app

my_app: main.cpp my_lib.cpp

	$(CXX) $(CXXFLAGS) main.cpp my_lib.cpp -o my_app

clean:

	rm -f my_app
```

Now look how it is so simple to use CMake instead

Go to your **CMakeLists.txt,** and type these lines:

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.10)

project(MyProject)

set(CMAKE_CXX_STANDARD 17)

set(CMAKE_CXX_STANDARD_REQUIRED ON)

set(CMAKE_CXX_CLANG_TIDY

   clang-tidy;

   -checks=cppcoreguidelines-*;

)

add_executable(my_app

main.cpp

my_lib.cpp

)
```

Just like that we complied files in 4 lines WOW

How to write CMakeLists.txt

First line is the Cmake minimum version required we will use 3.10 because most of old programs support that version. (We can use higher version but the higher we use, less programs will support).

```
cmake_minimum_required(VERSION 3.10)
```

Then we need to say the name of the project just name it whatever you want.

```
project(MyProjectName)
```

Then we **CAN add (not must)** define the C++ Version we want:

```
set(CMAKE_CXX_STANDARD 17)

set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

set(CMAKE_CXX_STANDARD 17):
tells CMake to compile the project using the **C++17** standard.
If we can set to 23 or 26 but we will define 17 as Version.

set(CMAKE_CXX_STANDARD_REQUIRED ON):
tells CMake that **C++17 is required**, so it should not use an older C++ version instead.

After that we will add the Clang-Tidy

```
set(CMAKE_CXX_CLANG_TIDY

   clang-tidy;

   -checks=cppcoreguidelines-*;

)
```

CMAKE_CXX_CLANG_TIDY tells CMake to run **Clang-Tidy** on the C++ source files during the build process.
In this example, we use clang-tidy with the checks group **cppcoreguidelines-***, which enables checks based on the C++ Core Guidelines.

There are many other ways to configure Clang-Tidy, and many additional options and check groups are **available in the official documentation**. However, in this course we compile relatively simple programs, so this line is enough and is a good choice for our needs.

Finally, we need to add our C++ source files:

```
add_executable(my_app

main.cpp

my_lib.cpp

)
```

add_executable() tells CMake to create an executable program.

- The first argument, my_app, is the name of the output program.
- The following arguments are the source files used to build it.
- One of these files must contain the main() function.
- You can add as many additional .cpp files as your project needs.

How I Compiled multiple programs ?

It is so simple just add another "**add_executable**" like this

CMakeList.txt

```
cmake_minimum_required(VERSION 3.10)

project(AutoProject CXX)

set(CMAKE_CXX_STANDARD 17)

set(CMAKE_CXX_STANDARD_REQUIRED ON)

set(CMAKE_CXX_CLANG_TIDY

  clang-tidy;

  -checks=cppcoreguidelines-*

)

add_executable(program_one program_one.cpp)

add_executable(program_two program_two.cpp)
```

We finished to use **CMakeLists.txt** 😄

Now we know how to use **CMakeLists.txt**

How to call CMake ?

Note: This method on Windows as far i know these should work on Mac too and Linux

I'm attaching a YouTube guide in case I say something that isn't exactly correct

I recommend checking what I'm saying for those who use Mac and Linux

setup Mac OS for C++, CMake development(build+debug) using Visual Stud...

Before I start

CMake choose the default generator for your system it does not mean you need to choose that, I will show you 2 ways
1) For default generator (most of you all use it) any OS (Each OS has it own default)
2) Ninja generator (Sometimes you may need it), Yes you need to download it from the internet and install it. Cross platform.

Why we want to use not the default generator ?

The Problem with "Default"

When you run CMake without specifying a generator, it picks the most common tool for your current OS:
the "Default" generator usually ties you to the specific operating system you are currently using, **which can cause headaches** when you want your code to run everywhere.

Operating System	Default Generator	Native Compiler
Windows	**Visual Studio**	**MSVC** (Microsoft Visual C++)
Linux	**Unix Makefiles**	**GCC** (GNU Compiler Collection)
macOS	**Xcode / Unix Makefiles**	**Apple Clang** (LLVM)

Why Use Ninja (or another specific tool) instead?

1. Consistent Behavior Using a specific toolset like GCC or Clang across all systems ensures your code behaves identically regardless of the host OS. This prevents "it works on my machine" bugs caused by different compilers interpreting the C++ standard differently.

2. Avoiding OS "Lock-in" Relying on a universal toolset prevents you from accidentally using proprietary, OS-specific features (like Windows-only or Mac-specific headers). It forces you to write truly portable code that can move between any system without breaking.

3. Matching the Production Environment Since most servers and Docker containers run Linux, using a matching compiler locally ensures your development environment mimics production. This stops "deployment day" errors where code works on your personal computer but fails once it is uploaded.

4. Tool & Linter Compatibility Professional tools like clang-tidy were built for the LLVM/GCC ecosystem and work most reliably when paired with those specific compilers. Using a cross-platform generator ensures these code-cleaning tools provide the same warnings to everyone on the team.

How to install Ninja generator

First go to this link: Releases · ninja-build/ninja

- Windows: ninja-win.zip

Download the latest version

For Windows:

- Download the file
- Go to your C:\Program Files
- Make a folder called "**Ninja**" (so we know it is ninja generator)
- **Add the exe file** to that folder
- Go to **edit environment variables**
- Go to Path and add: C:\Program Files\Ninja
- Open new Terminal and type: **ninja --version** if you see **1.13.2** you done

---

For macOS

The best way to install Ninja on a Mac is using a package manager:

- **Using Homebrew:** Open your terminal and run: **brew install ninja**
- **Using MacPorts:** Run: **sudo port install ninja**
- After installing, open a **new** terminal window and type: **ninja --version**

For Linux:

Most Linux distributions include Ninja in their official repositories:

- **Ubuntu / Debian / Mint:** Run: **sudo apt install ninja-build**

- After installing, open a **new** terminal window and type: **ninja --version**

How to call Cmake ?

- Go to your project folder
- Open terminal in that folder (or set path to that folder)

Default generator:

cmake -S . -B build

cmake --build build

Finding the executable program:

Windows:

Inside the build folder there will be a **Debug folder**, you can run using this command from the project folder

.\build\Debug\OurProgram.exe

For Linux / macOS:

Most often:

./build/OurProgram

Ninja:

cmake -S . -B build -G "Ninja"

cmake --build build

Finding the executable program:

Windows:

Inside build folder there will be the executable program (there are no Debug folder)

.\build\OurProgram.exe

For Linux / macOS:

Most often:

```
./build/OurProgram
```

Note about “Build” folder

We do **not have to** name the folder **build**.
The name is arbitrary, and CMake will use whatever folder we specify.

For example, instead of:

```
cmake -S . -B build
cmake --build build
```

we could use any other name:

```
cmake -S . -B bananananana
cmake --build bananananana
```

This would still work the same way. CMake would simply place all generated build files and compiled output inside the folder bananananana.

However, using the name **build** is a common convention because it clearly indicates that the folder contains the generated build files and compiled artifacts.